



UiT Norges arktiske universitet

Skoleeksamen i:	AUT-2602 Programmering med mikrokontroller
Dato og tidspunkt:	Klokka 09:00-13:00 22.11.2024
Målform:	Bokmål
Fagansvarlig (Navn og telefonnr.):	Per Anton Øverseth Olsen
Antall sider:	6 inkludert forside
Support	Du kan ringe 776 20 880 for support på eksamensdagen
Vekting av spørsmål, eller annen informasjon:	Besvarelsen kan være på norsk, engelsk eller en blanding.
Viktig informasjon om sitering og plagiering:	1. Dette er en individuell eksamen som skal besvares uten samarbeid med andre. 2. Alle hjelpemidler er tillatt (egne notater, pdfer fra forelesningene, lærebok, internett etc. Se Canvas eller uit.no for info om KI). 3. Alle eksamener som leveres i WISEFLOW blir automatisk sjekket for plagiat. Det er ikke tillatt å kopiere medstudenter, nettressurser, kilder, eller litteratur uten referanser.

Oppgave 1: Faktaspørsmål

I denne oppgaven skal dere svare så kort som mulig. Dersom det spørres om pinner, svar med PAo for PORTA, pinne o. Liten n indikerer heltall, og liten x indikerer bokstav, slik som PORTx.PINnCTRL.

a) 4%

Hva menes med "full duplex" og "half duplex" i USART-kommunikasjon med og uten RS485-funksjonalitet?

Svar: Full duplex tillater kommunikasjon i begge retninger samtidig, mens "half duplex" tillater kun i én retning om gagen.

b) 4%

Hva må stå i registeret "USARTn.BAUD" dersom baudrate skal være på 9600, i *asynchronous normal mode*, ordlengden er på 8-bit og CLK_PER = 12MHz?

Svar: For asynchronous normal mode brukes formelen:

$$\text{BAUD} = (64 * f_{\text{CLK_PER}}) / (S * f_{\text{BAUD}})$$

hvor $f_{\text{CLK_PER}} = 12\,000\,000\text{Hz}$, $S = 16$ og $f_{\text{BAUD}} = 9600$

Settes dette inn får vi:

$$\text{BAUD} = (64 * 12\,000\,000) / (16 * 9600) = 5000$$

Registeret "USARTn.BAUD" må stå på 5000.

c) 4%

Mange periferi-moduler på AVR128DB har registrene INTCTRL og INTFLAGS, med korresponderende bits (slik som OVF_bp). Hva er sammenhengen mellom disse to registertypene?

Svar: INTCTRL registeret blir brukt for å kontrollere hvilke avbrytelser som blir aktivert, mens INTFLAGS registeret endrer status for å indikerer indikerer tilstanden til avbrytelsene (om avbrytelsen har skjedd eller ikke).

d) 4%

Hvilke pinner kan benyttes til 8-bit-PWM dersom man bruker TCAo på AVR128DB48?

Svar: Pinnene PEO til PE2 can benyttes i dette tilfellet.

e) 4%

Hva er forskjellen på å implementere regelmessig ADC-lesing med og uten å bruke *event-routing* (EVSYS) og hvordan påvirker dette sleep-funksjonalitet?

Svar: Uten EVSYS må lesingen synkroniseres manuelt i programmet, noe som kan hindre mikrokontrolleren fra å gå inn i dyp søvnmodus. Ved å bruke EVSYS kan eksterne hendelser utløse lesinger, slik at mikrokontrolleren kan forbli i søvnmodus og oppnå lavere strømforbruk.

f) 3%

Hva er grunnen for at det anbefales å benytte *pull-up* på ubrukte pinner dersom man vil spare strøm på en mikrokontroller?

Svar: Pull-up-motstander forhindrer unødvendig strømforbruk ved at tilfeldig støy ikke tolkes som data.

g) 5%

Hvilke faktorer er viktige å tenke på dersom CLK_PER endres for et innebygd system med digitale og analoge sensorer tilkoblet?

Svar: Viktige faktorer å vurdere ved endring av CLK_PER inkluderer timing av kommunikasjon, timing av sensoravlesninger og strømforbruk.

h) 4%

Beskriv kort hvordan en AVR128DB kan lese av en verdi fra en tenkt ekstern sensor ved hjelp av SPI.

Svar: For å lese en verdi fra en ekstern sensor via Serial Peripheral Interface (SPI), konfigurerer AVR128DB48 SPI-modulen til å velge ønsket sensor ved hjelp av Client Select (SS). Deretter sender den en forespørsel om data gjennom Host Out Client In (MOSI)-linjen, og mottar til slutt data fra sensoren via Host In Client Out (MISO)-linjen.

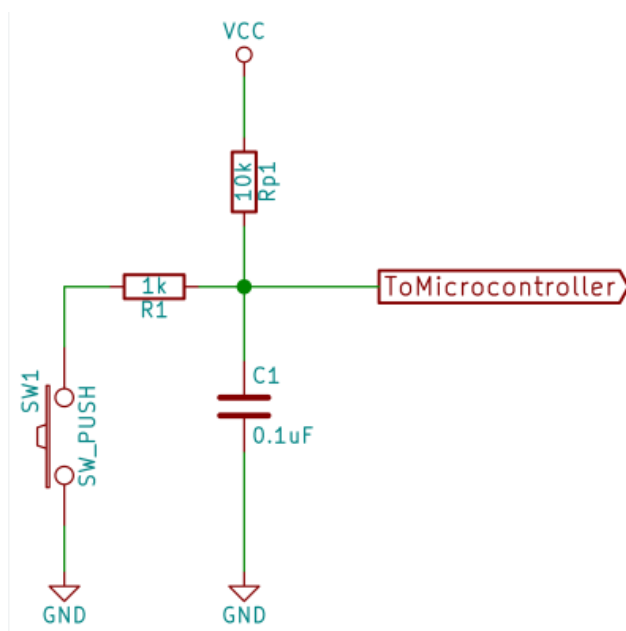
i) 4%

Forklar hovedforskjellen mellom å implementere ADC med *single conversion mode*, og *free-running mode*, hvis programavbrudd benyttes i begge tilfeller.

Svar: Single conversion mode utfører én ADC-konvertering før den stopper. Free-running mode fortsetter å utføre konverteringer til den stoppes manuelt. Ved å bruke avbrytelser er det mulig å benytte dataen umiddelbart etter hver konvertering.

j) 5%

SW1 er en typisk trykk-knapp. Hva er hensikten med en kondensatoren C1 og motstanden Rp1 i figur 1, og hvordan kan man med kode tilnærme seg samme effekt i mangel på kondensator?



Figur 1: Krets med knapp tilkoblet en pinne.

Svar: Kondensatoren C1 demper støy (debouncing), og RP1 er en pull-up-motstand. Dette kan gjenskapes i programmet ved å innføre en kort forsinkelse etter at input er mottatt, før noe mer skjer i koden.

k) 5%

Et program har mange forskjellige oppgaver som skal gjøres regelmessig. Hva er forskjellen mellom å bruke `_delay_ms()`, RTC med "overflow interrupt" og ISR-funksjon og TCA med "overflow interrupt" og ISR-funksjon?

Svar: `_delay_ms()` blokkerer CPUen mens den venter, RTC med "overflow interrupt" har lavere strømforbruk og tillater tidsavbrudd over lengre perioder og TCA med "overflow interrupt" tillater tidsavbrudd for kortere perioder og kan kjøre andre oppgaver parallelt.

Oppgave 2: Kodeforståelse og feilsøking

a) 7%

b) Beregn de manglende verdiene og forklar kodelinjene merket med ? i kodeutdraget nedenfor, slik at tonen som høres på buzzeren øker gradvis fra 500Hz til 1kHz på 4 sekunder og repeterer dette. Du kan også foreslå endringer hvis det er behov.

```
1 #include <avr.io.h>
2 #include <avr/interrupt.h>
3 #define F_CPU 4000000UL
4 #define BUZZER_PORT PORTF
5 #define BUZZER_PIN_bp 2
6 volatile uint16_t counter = 0; // ? = 0
7
8 ISR(TCA0_OVF_vect) {
9     BUZZER_PORT.OUTTGL = (1<<BUZZER_PIN_bp);
10    TCA0.SINGLE.INTFLAGS |= TCA_SINGLE_OVF_bm;
11 }
12 ISR(TCA1_OVF_vect) {
13     counter = (counter + 1)%4000; // ?_1 = 1 og ?_2 = 4000
14     TCA0.SINGLE.PERBUF = ? - (counter/100); // ? = 124// ?
15     TCA1.SINGLE.INTFLAGS |= TCA_SINGLE_OVF_bm;
16 }
17 int main(void)
18 {
19     usart_usb_init();
20     BUZZER_PORT.DIRSET = (1<<BUZZER_PIN_bp);
21     TCA0.SINGLE.CTRLA |= TCA_SINGLE_ENABLE_bm | ?; // ?=TCA_SINGLE_CLKSEL_DIV64_gc
22     TCA0.SINGLE.INTCTRL |= TCA_SINGLE_OVF_bm;
23     TCA0.SINGLE.PERBUF = ?; // ? = 124 // ?
24     TCA1.SINGLE.CTRLA |= TCA_SINGLE_ENABLE_bm | ?; // ?=TCA_SINGLE_CLKSEL_DIV64_gc
25     TCA1.SINGLE.INTCTRL |= TCA_SINGLE_OVF_bm;
26     TCA1.SINGLE.PERBUF = ?; // ? = 4000 //?
27     sei(); // ?
28 while(1) {
29 }
30 }
```

Svar:

```
TCA0.SINGLE.PERBUF = 124 - (counter/100);
```

Denne linjen justerer perioderegisteret PERBUF.

Ved å bruke formelen $PER = f_{CPU} / (DIV * F_{PWM}) = 4000000 / (64 * 500) = 125$

og $PER = 4000000 / (64 * 1000) = 62.5$. (rundes ned)

Det trekkes også fra 1 siden registeret starter på 0 og ikke 1.

```
TCA0.SINGLE.PERBUF = 124;
```

Denne linjen setter den initielle verdien av TCA0 sitt PERBUF som tilsvarer en PWM-frekvens på 500Hz.

```
TCA1.SINGLE.PERBUF = 4000;
```

Denne linjen setter den initielle verdien av TCA1 sitt PERBUF som styrer hvor ofte counter økes.

```
Sei();
```

Denne linjen aktiverer global avbruddshåndtering i mikrokontrolleren

c) 6%

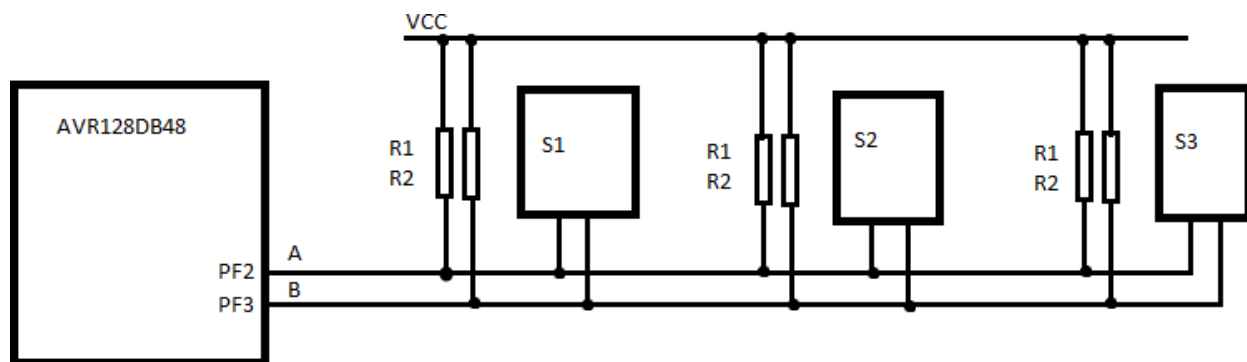
Forklar hva som skjer i de kommenterte kodelinjene og den generelle funksjonaliteten til funksjonen. Illustrer hvordan funksjonen brukes i kode.

```
1  uint8_t get_ADC_filtered(uint16_t oldVals, uint16_t *res, uint8_t filterSize){ // 1
2  if(filterSize == 0)
3      return 1;
4  while(!(ADC0.INTFLAGS & ADC_RESRDY_bm)); // 2
5  oldVals[filterSize-1] = ADC0.RES; // 3
6  uint16_t tmp = 0;
7  for(uint8_t i = 0; i < filterSize; i++){
8      tmp += oldVals[i]; // 4
9  }
10 *res=tmp/filterSize;
11 return 0; // 5
12 }
```

Svar: Denne funksjonen leser av en ADC-verdi, summerer versdiene og returnerer gjennomsnittet av disse. Linje 1 inneholder funksjonstypen, navnet og argumentene funksjonen mottar for å behandle verdiene. Linje 2 blokkerer programmet til ADC-konverteringen er fullført. Linje 3 lagrer ADC-resultatet i den siste posisjonen i det lokale bufferet («oldVals»). Linje 4 øker det summerte resultatet i et midlertidig buffer som legger til siste verdien i en loop gjennom ADC-verdi bufferet. Og linje 5 returnere 0 for å indikere at funksjonen er ferdig (verdien er lagret i res pekeren).

d) 5%

Hva kan man si om signalene på banene/lederne A og B ut i fra illustrasjonen i figur 2?



Figur 2: Illustrasjon av krets med mikrokontroller, motstander og tre sensorer

Svar: Motstandene er brukt som pull-up motstander. Mikrokontrollerens pinner PF2 og PF3 , er koblet til A og B. Sensorene vil sende signal til mikroprosessen ved endring til lavt nivå (VCC har høyt).

Oppgave 3: Programmering og implementasjon

I denne oppgaven skal du vise at du kan lage egen kode, tilpasse kode til andre formål og beskrive hvordan mikrokontrolleren kan brukes til å løse problemer. Bruk gjerne pseudokode dersom du ikke husker kodeordene ordrett. Du kan ta utgangspunkt i at koden som følger med i oppgavesettet kan brukes som om det var c-filer med tilhørende h-filer. Husk at du kan få poeng selv om bare deler av oppgaven er fullført.

Svar: Se vedlegg

a) 10%

Skriv et program som inkrementer et binært tall visualisert med lysdiodene DO-2 (3 stykk) ca. hvert sekund. Tallet starter på 0b000, inkrementeres helt til 0b111, og starter på nytt. Dersom knappene SW1 - 3 trykkes inn med samme konfigurasjon som lysene, skal mikrokontrolleren sende meldingen "suksess!" til en datamaskin ved å bruke USART. Forklar hvordan implementasjonen hadde sett ut dersom det også skulle benyttes søvnmodus. (Tips: CCL må deaktiveres mens det gjøres endringer).

b) 15%

Lag et program som foretar ADC lesinger med en frekvens på 10 Hz (relativt nøyaktig) fra meldingen "<n1>" mottas, til meldingen "<no>" mottas via USART, der n er et tall som symboliserer AINn. Dersom det går mer enn 10 sekunder før "<no>" mottas, skal også avlesingene slutte. Deretter skal alle verdiene returneres til datamaskinen med USART.

c) 15%

En vifte med PWM-styring og takometer brukes til å kjøle ned en komponent med en analog temperatursensor. Hastigheten på viften justeres mellom 1ms og 2ms med PWM-frekvens på 50Hz. Temperatursensoren er koblet på PD4 og gir spenninger mellom 0V og 2.5V der relasjonen mellom temperatur og spenning er gitt av: $T \approx (30 + 20 \times V_{sens})^{\circ}\text{C}$. Lag et program som justerer blåsing mellom minimum og maksimum hastighet fra $T = 50^{\circ}\text{C}$ til $T = 80^{\circ}\text{C}$. Foreslå en måte å måle om hastigheten på vifta samsvarer med pådraget/PWM-verdien.

Lykke til!